

1. Departments (D)

| Column Name | Data Type   | Constraint  | Description                  |
|-------------|-------------|-------------|------------------------------|
| dept_id     | INT         | PRIMARY KEY | Unique ID for the department |
| dept_name   | VARCHAR(50) | NOT NULL    | Name of the department       |
| location_id | VARCHAR(50) | NOT NULL    | Geographic location          |

Sample Data for Departments

| dept_id | dept_name   | location_id |
|---------|-------------|-------------|
| 10      | Consulting  | NYC         |
| 20      | Engineering | SF          |
| 30      | R&D         | London      |
| 40      | Sales       | NYC         |
| 50      | HR          | SF          |
| 60      | Admin       | London      |

---

2. Employees (E)

| Column Name   | Data Type     | Constraint  | Description                |
|---------------|---------------|-------------|----------------------------|
| employee_id   | INT           | PRIMARY KEY | Unique ID for the employee |
| employee_name | VARCHAR(50)   | NOT NULL    | Employee full name         |
| manager_id    | INT           | FOREIGN KEY | ID of the manager          |
| dept_id       | INT           | FOREIGN KEY | Department ID              |
| salary        | DECIMAL(10,2) | NOT NULL    | Annual salary              |

Sample Data for Employees

| employee_id | employee_name     | manager_id | dept_id | salary    |
|-------------|-------------------|------------|---------|-----------|
| 100         | Alice Smith (CEO) | NULL       | 10      | 180000.00 |
| 101         | Bob Johnson       | 100        | 20      | 120000.00 |

| employee_id | employee_name | manager_id | dept_id | salary    |
|-------------|---------------|------------|---------|-----------|
| 102         | Charlie Brown | 101        | 20      | 95000.00  |
| 103         | Diana Prince  | 100        | 30      | 110000.00 |
| 104         | Evan Clark    | 103        | 30      | 85000.00  |
| 105         | Fiona Glen    | 101        | 20      | 90000.00  |
| 106         | George King   | 100        | 40      | 75000.00  |
| 107         | Hannah Lee    | 103        | 30      | 88000.00  |
| 108         | Ivan Rossi    | 100        | 50      | 60000.00  |
| 109         | Jane Doe      | 101        | 20      | 125000.00 |
| 110         | Kevin Blue    | 104        | 30      | 86000.00  |

### 3. Projects (P)

| Column Name  | Data Type   | Constraint  | Description               |
|--------------|-------------|-------------|---------------------------|
| project_id   | INT         | PRIMARY KEY | Unique ID for the project |
| project_name | VARCHAR(50) | NOT NULL    | Project name              |
| client_name  | VARCHAR(50) | NOT NULL    | Client name               |
| start_date   | DATE        | NOT NULL    | Project start date        |
| dept_id      | INT         | FOREIGN KEY | Department responsible    |

#### Sample Data for Projects

| project_id | project_name      | client_name          | start_date | dept_id |
|------------|-------------------|----------------------|------------|---------|
| 5001       | Aurora Migration  | GlobalTech Solutions | 2023-01-15 | 20      |
| 5002       | Synergy App Dev   | Zenith Corp          | 2023-05-20 | 30      |
| 5003       | Q4 Sales Strategy | Apex Industries      | 2024-01-01 | 40      |
| 5004       | Global Expansion  | GlobalTech Solutions | 2024-03-10 | 10      |
| 5005       | Internal HR Tool  | Internal             | 2024-02-01 | 50      |

| project_id | project_name    | client_name | start_date | dept_id |
|------------|-----------------|-------------|------------|---------|
| 5006       | Data Analytics  | Zenith Corp | 2022-11-01 | 20      |
| 5007       | Future Tech R&D | R&D Dept    | 2024-06-01 | 30      |

4. Assignments (A)

| Column Name   | Data Type | Constraint  | Description           |
|---------------|-----------|-------------|-----------------------|
| assignment_id | INT       | PRIMARY KEY | Unique assignment ID  |
| employee_id   | INT       | FOREIGN KEY | Employee assigned     |
| project_id    | INT       | FOREIGN KEY | Project assigned      |
| hours_worked  | INT       | NOT NULL    | Hours worked          |
| start_date    | DATE      | NOT NULL    | Assignment start date |

Sample Data for Assignments

| assignment_id | employee_id | project_id | hours_worked | start_date |
|---------------|-------------|------------|--------------|------------|
| 1             | 101         | 5001       | 150          | 2023-01-15 |
| 2             | 102         | 5001       | 160          | 2023-01-15 |
| 3             | 105         | 5001       | 140          | 2023-02-01 |
| 4             | 103         | 5002       | 80           | 2023-05-20 |
| 5             | 104         | 5002       | 70           | 2023-06-01 |
| 6             | 106         | 5003       | 5            | 2024-01-01 |
| 7             | 100         | 5004       | 20           | 2024-03-10 |
| 8             | 107         | 5002       | 75           | 2023-05-25 |
| 9             | 109         | 5006       | 200          | 2022-11-01 |
| 10            | 101         | 5006       | 250          | 2022-11-01 |
| 11            | 102         | 5006       | 180          | 2022-12-01 |
| 12            | 110         | 5007       | 10           | 2024-06-01 |

---

## Database Schema Structure

| Table Name      | Purpose                     | Key Columns                                           |
|-----------------|-----------------------------|-------------------------------------------------------|
| Employees (E)   | Stores employee information | employee_id (PK), manager_id (FK), dept_id (FK)       |
| Departments (D) | Stores department details   | dept_id (PK), location_id                             |
| Projects (P)    | Stores project details      | project_id (PK), client_name                          |
| Assignments (A) | Links employees to projects | assignment_id (PK), employee_id (FK), project_id (FK) |

---

## SQL Questions

1.

List the Employee Name, their Department Name, and the Location Name for all employees.

```
SELECT e.employee_name, d.dept_name, d.location_id
FROM Employees e
JOIN Departments d
ON e.dept_id = d.dept_id;
```

---

2.

Retrieve the Project Name, Client Name, and the Total Hours Worked on that project, but only for projects managed by the 'Consulting' department.

```
SELECT p.project_name, p.client_name,
SUM(a.hours_worked) AS total_hours
FROM Projects p
JOIN Assignments a
ON p.project_id = a.project_id
JOIN Departments d
ON p.dept_id = d.dept_id
WHERE d.dept_name = 'Consulting'
GROUP BY p.project_name, p.client_name;
```

---

**3.**

Find the names of Employees who have worked on a project with Client Name 'GlobalTech Solutions'. List the Employee Name and Project Name.

```
SELECT e.employee_name, p.project_name
FROM Employees e
JOIN Assignments a
ON e.employee_id = a.employee_id
JOIN Projects p
ON a.project_id = p.project_id
WHERE p.client_name = 'GlobalTech Solutions';
```

---

**4.**

List all Departments that have no Employees currently assigned to them.

```
SELECT d.dept_name
FROM Departments d
LEFT JOIN Employees e
ON d.dept_id = e.dept_id
WHERE e.employee_id IS NULL;
```

---

**5.**

Show the Manager Name, the Department Name they manage, and the total number of direct reports they have.

---

### **Aggregation and Grouping**

**6.**

Calculate the Average Hours Worked per Employee across all projects. List the Employee Name and the average hours.

```
SELECT e.employee_name,
AVG(a.hours_worked) AS avg_hours
FROM Employees e
JOIN Assignments a
ON e.employee_id = a.employee_id
GROUP BY e.employee_name;
```

---

**7.**

Find the Department Name(s) that have an average salary above the company-wide average salary.

```
SELECT d.dept_name
FROM Employees e
JOIN Departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > (
SELECT AVG(salary)
FROM Employees
);
```

---

**8.**

Identify the Client Name(s) whose projects have a total combined hours worked exceeding 500 hours.

```
SELECT p.client_name,
SUM(a.hours_worked) AS total_hours
FROM Projects p
JOIN Assignments a
ON p.project_id = a.project_id
GROUP BY p.client_name
HAVING SUM(a.hours_worked) > 500;
```

---

**9.**

For each Location, find the Department Name with the highest count of employees.

---

**10.**

Calculate the total number of employees assigned to each Project, but only include projects that have more than 5 employees assigned.

```
SELECT p.project_name,
COUNT(DISTINCT a.employee_id) AS total_employees
FROM Projects p
JOIN Assignments a
ON p.project_id = a.project_id
```

```
GROUP BY p.project_name
HAVING COUNT(DISTINCT a.employee_id) > 5;
```

---

## **Advanced Filtering**

### **11.**

Find the Employee Name(s) who work on ALL projects managed by the 'R&D' department.

---

### **12.**

Retrieve the Department Name(s) that have employees assigned to more than one project.

```
SELECT DISTINCT d.dept_name
FROM Departments d
JOIN Employees e
ON d.dept_id = e.dept_id
JOIN Assignments a
ON e.employee_id = a.employee_id
GROUP BY d.dept_name, e.employee_id
HAVING COUNT(DISTINCT a.project_id) > 1;
```

---

### **13.**

Write a query to list the Employee Name and Project Name for any employee who has worked less than 10 hours on a project.

```
SELECT e.employee_name, p.project_name
FROM Employees e
JOIN Assignments a
ON e.employee_id = a.employee_id
JOIN Projects p
ON a.project_id = p.project_id
WHERE a.hours_worked < 10;
```

---

### **14.**

Find the Project Name(s) that have no assigned employees.

```
SELECT p.project_name
FROM Projects p
LEFT JOIN Assignments a
ON p.project_id = a.project_id
WHERE a.assignment_id IS NULL;
```

---

**15.**

Select the Employee Name(s) whose salary is greater than the maximum salary in their own department.

```
SELECT employee_name
FROM Employees e1
WHERE salary > (
SELECT MAX(e2.salary)
FROM Employees e2
WHERE e1.dept_id = e2.dept_id
AND e1.employee_id <> e2.employee_id
);
```

---

### **Dates and MySQL Specific Functions**

**16.**

Assuming the Assignments table has a start\_date column, find the Employee Name and their oldest project start date.

```
SELECT e.employee_name,
MIN(a.start_date) AS oldest_project_date
FROM Employees e
JOIN Assignments a
ON e.employee_id = a.employee_id
GROUP BY e.employee_name;
```

---

**17.**

Write a MySQL query using the DATE\_FORMAT function to list all projects that were active in the year 2023.

```
SELECT project_name,
start_date
```



```
FROM Projects
WHERE DATE_FORMAT(start_date, '%Y') = '2023';
```

---

**18.**

Use the MySQL-specific IFNULL function to list all Employee Names. If an employee has no manager, display "CEO" instead of NULL.

```
SELECT employee_name,
IFNULL(manager_id, 'CEO') AS manager
FROM Employees;
```

---

**19.**

Find the top 3 Departments based on the total hours worked by all their employees across all projects.

**20.**

Display the Project Name and a status ("High Activity" if total hours > 300, "Normal" otherwise) using the CASE statement.

```
SELECT p.project_name,
CASE
WHEN SUM(a.hours_worked) > 300 THEN 'High Activity'
ELSE 'Normal'
END AS status
FROM Projects p
LEFT JOIN Assignments a
ON p.project_id = a.project_id
GROUP BY p.project_name;
```

---

## **Complex Scenarios and Performance**

**21.**

List the Employee Name and their Manager's Name, even if the employee has no manager, or if a potential manager has no employees reporting to them.

```
SELECT e.employee_name,  
m.employee_name AS manager_name  
FROM Employees e  
LEFT JOIN Employees m  
ON e.manager_id = m.employee_id;
```

---

**22.**

Write a query using a Common Table Expression (CTE) to find the Project Name and the number of unique employees assigned to it.

---

**23.**

Find the Employee Name(s) who are assigned to the same projects as Employee ID 101.